

Taller de analítica en los negocios

Analítica en marketing



Escuela de Administración y Gestión Empresarial

Directora: Lorena Baus Piva

Elaboración

Experto disciplinar: Tomás von Bischoffshausen Gariazzo

Diseñador instruccional: Óscar González Cantin

Editor instruccional: David Villagrán Ruz

Validación

Experto disciplinar: Andrés Montecinos Rùth

Jefa de diseño instruccional: Sandra Betancur Cordero

Equipo de desarrollo

AIEP

AÑO

2025

Tabla de contenidos

Aprendizaje esperado de la semana.....	5
Introducción	6
1. Análisis del modelado de campañas de marketing.....	6
2. Preparación y limpieza de datos	7
2.1. Tratamiento de valores nulos y outliers.....	7
2.2. Codificación de variables categóricas	8
2.3. Normalización/estandarización de datos.....	9
3. Selección de variables.....	10
3.1. Criterio experto	10
3.2. Multicolinealidad en variables predictoras.....	10
4. Modelamiento de algoritmos regresores	11
4.1. Regresión lineal	11
4.2. Regresión polinómica.....	12
4.3. Regresión Ridge.....	13
4.4. Regresión Lasso.....	13
4.5. Comparación y aplicación práctica	14
5. Técnicas de evaluación y validación	15
5.1. Validación cruzada (Cross-validation)	15
5.2. RMSE (Root Mean Squared Error).....	16
5.3. MAE (Mean Absolute Error)	16

5.4. R-squared (R^2)	17
5.5. Gráfico de residuales y análisis de errores	17
6. Mejoramiento del modelo	19
6.1. Ajuste de hiperparámetros	19
Cierre	20
Referencias	21

Aprendizaje esperado de la semana

Aplican modelos predictivos en el contexto de una campaña de marketing, con algoritmos regresores a través de herramientas Python, considerando análisis de datos en el contexto de estrategias de marketing.



Fuente: Envato Elements (s. f.)

Introducción

¿Qué tan preciso es un modelo predictivo para resolver problemas del mundo real?

La respuesta depende de cómo se evalúa y mejora dicho modelo. En este apunte, exploraremos diversas técnicas de evaluación y validación, como la validación cruzada, el RMSE y el análisis de residuos, que permiten medir el desempeño del modelo. También profundizaremos en estrategias de mejoramiento, como el ajuste de hiperparámetros, esenciales para optimizar la precisión, minimizar errores y mejorar la capacidad del modelo para generalizar a nuevos datos.

1. Análisis del modelado de campañas de marketing

El análisis y modelado de campañas de marketing se ha consolidado como una estrategia clave para mejorar el rendimiento de las iniciativas comerciales. La literatura especializada destaca que, mediante técnicas de segmentación y predicción, las empresas pueden personalizar sus mensajes y mejorar la tasa de conversión. Modelos como el análisis de clústeres, la regresión logística y los árboles de decisión son frecuentemente aplicados para identificar patrones en el comportamiento del consumidor, permitiendo ajustar campañas de acuerdo con preferencias y necesidades específicas.

Un enfoque recurrente en los estudios es el uso de datos históricos de campañas anteriores, como clics, interacciones, tasas de apertura de correos y compras. Estos datos se procesan para entrenar modelos predictivos que determinen qué tipo de contenido o segmentación genera mejores resultados. La literatura también resalta la importancia de la preparación y calidad de los datos, ya que errores o inconsistencias pueden afectar negativamente el desempeño del modelo.

2. Preparación y limpieza de datos

Una campaña de marketing efectiva requiere un tratamiento riguroso de los datos para garantizar que los modelos predictivos produzcan resultados confiables. A continuación, se describen los pasos fundamentales para preparar los datos.

2.1. Tratamiento de valores nulos y *outliers*

Valores nulos: los datos faltantes son comunes en grandes conjuntos de datos y deben ser gestionados para evitar errores en el modelo.

Opciones de tratamiento:

1. **Imputación:** rellenar los valores faltantes con una medida central (promedio o mediana).

```
df['variable'].fillna(df['variable'].mean(), inplace=True)
```

2. **Eliminación:** si los datos faltantes son mínimos, se pueden eliminar las filas afectadas.

```
df = df.dropna()
```


Outliers: los valores atípicos pueden sesgar los resultados si no se gestionan adecuadamente.

Opciones de tratamiento:

1. **Winsorización:** limita los valores extremos dentro de un rango aceptable.

```
import numpy as np
```

```
df['variable'] = np.clip(df['variable'], lower_limit, upper_limit)
```

Visualización: utiliza gráficos de caja para identificar *outliers*.

```
df['variable'].plot.box()
```

2.2. Codificación de variables categóricas

Las variables categóricas, como el tipo de cliente o la región, deben ser transformadas en formato numérico para que los modelos predictivos puedan procesarlas.

Tipos de codificación:

1. **One-Hot Encoding:** crea una nueva columna para cada categoría.

```
df = pd.get_dummies(df, columns=['variable_categórica'])
```

2. **Label Encoding:** asigna un valor numérico a cada categoría.

```
from sklearn.preprocessing import LabelEncoder
```

```
encoder = LabelEncoder()
```

```
df['variable_categórica'] = encoder.fit_transform(df['variable_categórica'])
```

Recomendación: utiliza One-Hot Encoding si no hay un orden implícito en las categorías y Label Encoding si existe una jerarquía natural.

2.3. Normalización/estandarización de datos

Los modelos de machine learning, especialmente aquellos sensibles a la escala, como KNN y SVM, requieren que las variables estén en rangos comparables.

- **Normalización:** transforma los datos para que estén entre 0 y 1.

```
from sklearn.preprocessing import MinMaxScaler  
scaler = MinMaxScaler()  
df[['variable1', 'variable2']] = scaler.fit_transform(df[['variable1', 'variable2']])
```

- **Estandarización:** centra los datos en 0 y ajusta la varianza a 1.

```
from sklearn.preprocessing import StandardScaler  
scaler = StandardScaler()  
df[['variable1', 'variable2']] = scaler.fit_transform(df[['variable1', 'variable2']])
```

Resultado esperado: datos preparados y listos para el modelado, libres de inconsistencias y con una representación numérica adecuada.

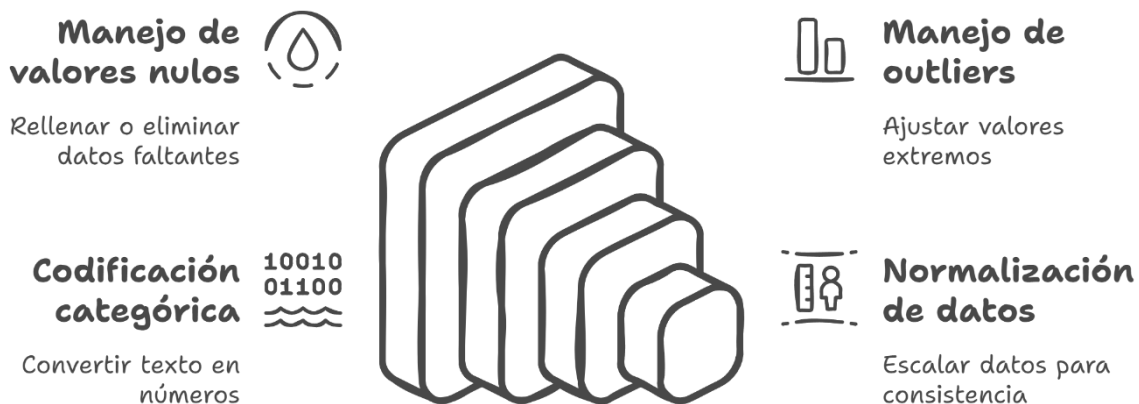


Figura 1. Proceso de preparación de datos para campañas de marketing

3. Selección de variables

La selección de variables es un paso crucial en la preparación de datos, ya que permite identificar cuáles son más relevantes para mejorar la precisión y generalización del modelo. Este proceso evita el sobreajuste y reduce el tiempo de procesamiento.

3.1. Criterio experto

El criterio experto implica la selección de variables basándose en el conocimiento del dominio o contexto del problema. Los expertos en marketing, por ejemplo, pueden identificar que variables como "frecuencia de compra" o "nivel de ingresos" son determinantes para una campaña, descartando aquellas que no aportan información valiosa.

Ventajas:

- Reduce el tiempo de procesamiento al enfocarse solo en variables clave.
- Mejora la interpretabilidad del modelo.

Aplicación:

Realiza una revisión inicial del conjunto de datos junto con expertos del área para identificar variables esenciales. Complementa esta selección con análisis estadísticos o gráficos exploratorios.

3.2. Multicolinealidad en variables predictoras

La multicolinealidad ocurre cuando dos o más variables independientes están altamente correlacionadas, lo que dificulta que el modelo distinga su

impacto individual en la variable objetivo. Esto puede afectar la estabilidad de los coeficientes y su interpretación en modelos de regresión.

Identificación: calcula la matriz de correlación entre las variables predictoras.

```
import seaborn as sns
import matplotlib.pyplot as plt
```

```
correlation_matrix = df.corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Matriz de correlación')
plt.show()
```

Tratamiento:

1. Elimina una de las variables altamente correlacionadas.
2. Utiliza técnicas como la regresión Ridge o Lasso para manejar la multicolinealidad.

4. Modelamiento de algoritmos regresores

Los algoritmos de regresión son utilizados para predecir variables continuas. A continuación, exploraremos diferentes técnicas de regresión.

4.1. Regresión lineal

La regresión lineal ajusta una línea recta que modela la relación entre una variable independiente y una dependiente. Su ecuación es:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

Implementación en Python:

```
from sklearn.linear_model import LinearRegression
```

```
modelo_lineal = LinearRegression()
```

```
modelo_lineal.fit(X_train, y_train)
```

Evaluación:

```
r2_score = modelo_lineal.score(X_test, y_test)
```

```
print(f"R²: {r2_score}")
```

4.2. Regresión polinómica

La regresión polinómica extiende el modelo lineal incorporando términos de orden superior para capturar relaciones no lineales.

Implementación en Python:

```
from sklearn.preprocessing import PolynomialFeatures
```

```
from sklearn.linear_model import LinearRegression
```

```
poly = PolynomialFeatures(degree=2)
```

```
X_poly = poly.fit_transform(X_train)
```

```
modelo_polinomico = LinearRegression()
```

```
modelo_polinomico.fit(X_poly, y_train)
```

Visualización:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
plt.scatter(X_train, y_train, color='blue')
plt.plot(X_train, modelo_polinomico.predict(X_poly), color='red')
plt.title('Regresión polinómica')
plt.show()
```

4.3. Regresión Ridge

La regresión Ridge es una técnica de regularización que penaliza los coeficientes grandes, lo que ayuda a controlar el sobreajuste y a manejar la multicolinealidad. La función objetivo incluye un término de penalización basado en la norma L2.

Implementación en Python:

```
from sklearn.linear_model import Ridge
```

```
modelo_ridge = Ridge(alpha=1.0)
modelo_ridge.fit(X_train, y_train)
```

Evaluación:

```
r2_ridge = modelo_ridge.score(X_test, y_test)
print(f"R2 (Ridge): {r2_ridge}")
```

4.4. Regresión Lasso

La regresión Lasso, similar a Ridge, agrega un término de penalización basado en la norma L1, lo que puede llevar a que algunos coeficientes sean exactamente cero, seleccionando automáticamente características relevantes.

Implementación en Python:

```
from sklearn.linear_model import Lasso
```

```

modelo_lasso = Lasso(alpha=0.1)
modelo_lasso.fit(X_train, y_train)

```

Evaluación:

```

r2_lasso = modelo_lasso.score(X_test, y_test)
print(f"R² (Lasso): {r2_lasso}")

```

Visualización de coeficientes:

```

import pandas as pd

coeficientes = modelo_lasso.coef_
coef_df = pd.DataFrame({'Característica': X_train.columns, 'Coeficiente': coeficientes})
coef_df.plot.bar(x='Característica', y='Coeficiente', title='Coeficientes en regresión Lasso')
plt.show()

```

4.5. Comparación y aplicación práctica

Algoritmo	Ventajas	Limitaciones
Regresión lineal	Simple, fácil de interpretar	No captura relaciones no lineales
Regresión polinómica	Captura relaciones no lineales	Propenso al sobreajuste
Regresión Ridge	Maneja multicolinealidad, reduce sobreajuste	No selecciona automáticamente características
Regresión Lasso	Selecciona características relevantes	Puede eliminar variables importantes

Tabla 1. Comparación entre técnicas de regresión

Aplicación práctica:

1. Selecciona el tipo de regresión en función de la complejidad del problema.

2. Entrena y evalúa diferentes modelos para comparar su rendimiento.
3. Ajusta los hiperparámetros para maximizar la precisión y generalización.

5. Técnicas de evaluación y validación

Para determinar la precisión y eficacia de los modelos de regresión, es importante utilizar técnicas de evaluación que midan los errores de predicción y el ajuste general del modelo. A continuación, se explican las métricas más comunes y la validación cruzada.

5.1. Validación cruzada (*Cross-validation*)

La validación cruzada es una técnica que consiste en dividir los datos en varios pliegues o particiones. El modelo se entrena en diferentes subconjuntos de los datos y se evalúa en el resto, obteniendo una medida promedio del rendimiento.

Implementación en Python:

```
from sklearn.model_selection import cross_val_score

scores = cross_val_score(modelo, X, y, cv=5, scoring='neg_mean_squared_error')
rmse_scores = (-scores) ** 0.5
print(f"RMSE promedio: {rmse_scores.mean()}")
```

Ventajas:

- Reduce el riesgo de sobreajuste.
- Proporciona una evaluación más estable del rendimiento.

5.2. RMSE (Root Mean Squared Error)

El RMSE es una medida de la magnitud del error de las predicciones. Se calcula como la raíz cuadrada del promedio de los errores al cuadrado. Cuanto menor sea el valor, mejor es el ajuste del modelo.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

Implementación:

```
from sklearn.metrics import mean_squared_error

rmse = mean_squared_error(y_test, y_pred, squared=False)
print(f"RMSE: {rmse}")
```

5.3. MAE (Mean Absolute Error)

El MAE mide el promedio de los errores absolutos. A diferencia del RMSE, no penaliza los errores grandes de manera desproporcionada.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

Implementación:

```
from sklearn.metrics import mean_absolute_error

mae = mean_absolute_error(y_test, y_pred)
print(f"MAE: {mae}")
```

5.4. R-squared (R^2)

El R^2 indica qué porcentaje de la variabilidad de la variable dependiente es explicado por el modelo. Un valor cercano a 1 implica un buen ajuste.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Implementación:

```
from sklearn.metrics import r2_score
```

```
r2 = r2_score(y_test, y_pred)
print(f"R²: {r2}")
```

5.5. Gráfico de residuales y análisis de errores

El análisis de los residuales permite verificar si el modelo cumple con las suposiciones de linealidad y homocedasticidad. Los residuales deben distribuirse aleatoriamente alrededor de cero.

Implementación de gráfico de residuales:

```
import matplotlib.pyplot as plt

residuales = y_test - y_pred
plt.scatter(y_pred, residuales)
plt.axhline(y=0, color='red', linestyle='--')
plt.title('Gráfico de residuales')
plt.xlabel('Predicciones')
plt.ylabel('Residuales')
plt.show()
```

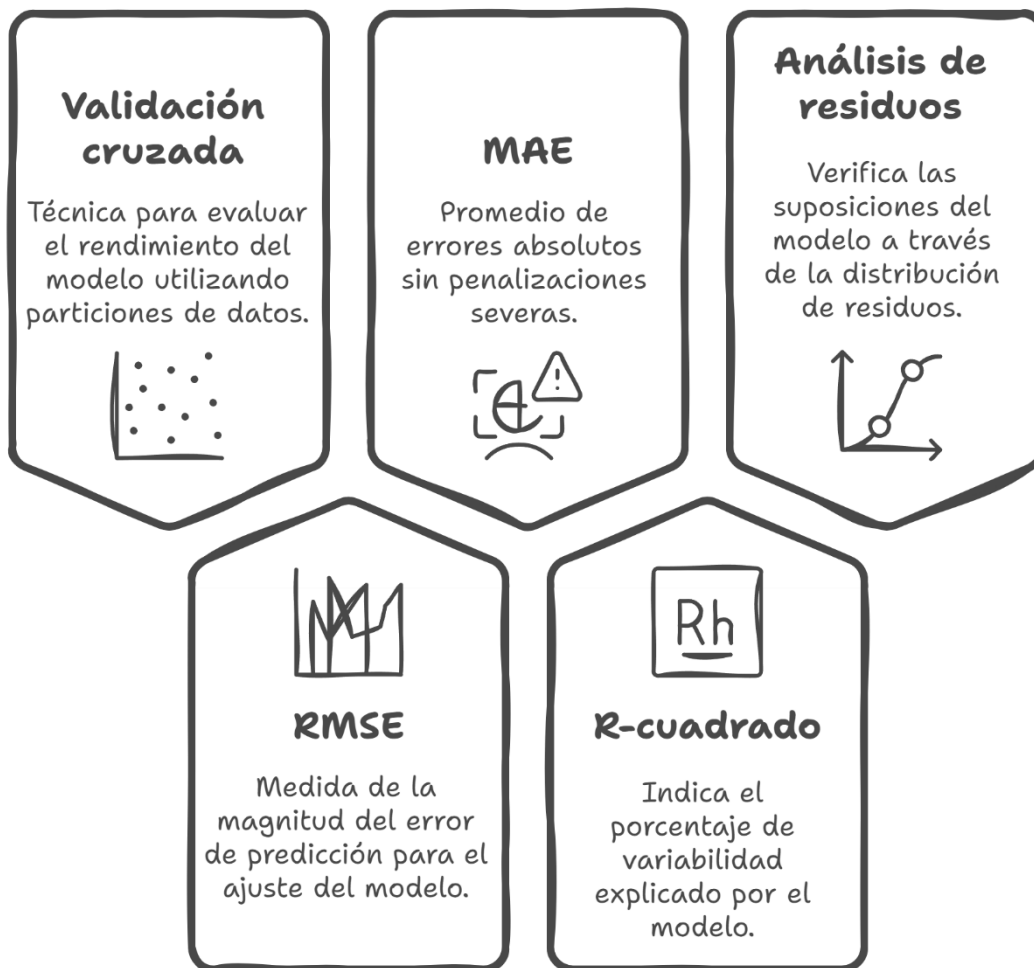


Figura 2. Técnicas de evaluación de modelos de regresión

6. Mejoramiento del modelo

6.1. Ajuste de hiperparámetros

El ajuste de hiperparámetros permite mejorar el rendimiento del modelo modificando configuraciones como la regularización en regresiones Ridge y Lasso, o el grado del polinomio en la regresión polinómica.

Ejemplo con Grid Search:

```
from sklearn.model_selection import GridSearchCV
from sklearn.linear_model import Ridge

parametros = {'alpha': [0.01, 0.1, 1.0, 10.0]}
grid_search = GridSearchCV(Ridge(), parametros, cv=5,
scoring='neg_mean_squared_error')
grid_search.fit(X_train, y_train)

print(f"Mejores hiperparámetros: {grid_search.best_params_}")
print(f"Mejor RMSE: {(-grid_search.best_score_) ** 0.5}")
```

Visualización:

```
resultados = pd.DataFrame(grid_search.cv_results_)
resultados.plot('param_alpha', 'mean_test_score', marker='o')
plt.title('Impacto del ajuste de hiperparámetros')
plt.xlabel('Valor de alpha')
plt.ylabel('Error medio')
plt.show()
```

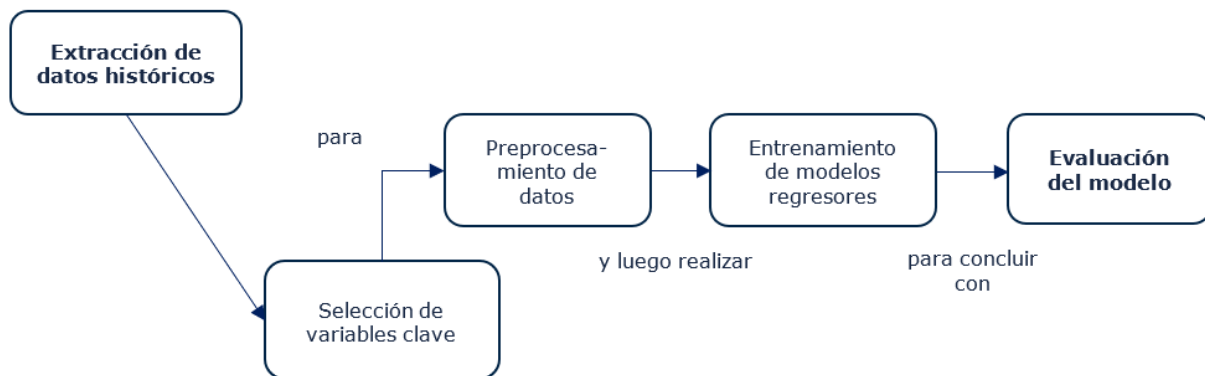
Ventajas:

- Mejora la precisión y estabilidad del modelo.

- Reduce el riesgo de sobreajuste al encontrar el equilibrio adecuado entre complejidad y ajuste.

Cierre

Por medio del siguiente organizador gráfico se destacan las ideas clave de esta semana:



La evaluación rigurosa y el ajuste adecuado del modelo son pasos esenciales para garantizar su eficacia en entornos reales. Métricas como el RMSE, el MAE y el R-cuadrado proporcionan una visión integral del rendimiento del modelo, mientras que el análisis de residuos permite detectar posibles limitaciones en su ajuste. Al aplicar técnicas de mejora, como la validación cruzada y el ajuste de hiperparámetros, es posible maximizar el potencial predictivo del modelo y mejorar la toma de decisiones basada en datos. Con estas herramientas, podrás construir modelos más robustos, confiables y precisos.

Referencias

Envato Elements (s. f.). Visión desde un ángulo elevado de los trabajadores.

[Imagen]. <https://elements.envato.com/es/high-angle-view-on-working-people-VK423DJ>

(s. f.) Análisis financiero, concepto de investigación de indicadores económicos [Imagen]. <https://elements.envato.com/es/finance-analytics-research-of-economic-indicators--9DQPGG6>

Las figuras (diagramas e imágenes) utilizadas en este apunte fueron generadas mediante Napkin (<https://www.napkin.ai/>)